

Experiment 05:

7. Laboratory Exercise

Code:

```
# Extended Euclidean Algorithm
def extended_euclidean_algorithm(first_number, second_number):
    if second_number == 0:
        return first_number, 1, 0

    greatest_common_divisor, first_coefficient, second_coefficient = (
        extended_euclidean_algorithm(
            second_number,
            first_number % second_number
        )
    )

    new_first_coefficient = second_coefficient
    new_second_coefficient = (
        first_coefficient
        - (first_number // second_number) * second_coefficient
    )

    return (
        greatest_common_divisor,
        new_first_coefficient,
        new_second_coefficient
    )

# ----- KEY GENERATION -----

prime_number_p = int(input("Enter prime number p: "))
prime_number_q = int(input("Enter prime number q: "))
public_exponent = int(input("Enter public key e: "))

# Calculate n = p x q
modulus_n = prime_number_p * prime_number_q

# Calculate phi(n) = (p - 1)(q - 1)
phi_n = (prime_number_p - 1) * (prime_number_q - 1)

# Check whether e and phi(n) are coprime
greatest_common_divisor, _, _ = extended_euclidean_algorithm(
    public_exponent,
    phi_n
)

if greatest_common_divisor != 1:
    print("Invalid value of e.")
    print("e and phi(n) must be coprime.")
    exit()

# Calculate d using Extended Euclidean Algorithm
_, private_exponent, _ = extended_euclidean_algorithm(
```

```
        public_exponent,
        phi_n
    )

# Make d positive
private_exponent = private_exponent % phi_n

# Display keys
print("\n----- KEY GENERATION -----")
print("n =", modulus_n)
print("phi(n) =", phi_n)
print("Public Key =", (public_exponent, modulus_n))
print("Private Key =", (private_exponent, modulus_n))

# ----- DIGITAL SIGNATURE GENERATION -----

message = int(input("\nEnter message M: "))

if message >= modulus_n:
    print("Message must be less than n.")
    exit()

# RSA Digital Signature Formula:
#  $S = M^d \bmod n$ 
digital_signature = (
    message ** private_exponent
) % modulus_n

print("\n----- DIGITAL SIGNATURE -----")
print("Message =", message)
print("Digital Signature =", digital_signature)

# ----- SIGNATURE VERIFICATION -----

received_signature = int(input("\nEnter signature S: "))
received_message = int(input("Enter message M: "))

# RSA Signature Verification Formula:
#  $M' = S^e \bmod n$ 
verified_message = (
    received_signature ** public_exponent
) % modulus_n

print("\n----- SIGNATURE VERIFICATION -----")
print("Signature =", received_signature)
print("Message =", received_message)
print("Calculated M' =", verified_message)

# Compare M' with the original message M
if verified_message == received_message:
    print("The message is authenticate")
else:
    print("Message is altered, Discard.")
```

1. Test the output of program for following exercises:

- a. For $p=11$, $q=3$ and $e=3$, find public and private keys using the RSA algorithm. For message, $M=10$, Find the digital signature value. Also show how any receiver of this signature will verify it.

Output:

```
Enter prime number p: 11
Enter prime number q: 3
Enter public key e: 3

----- KEY GENERATION -----
n = 33
phi(n) = 20
Public Key = (3, 33)
Private Key = (7, 33)

Enter message M: 10

----- DIGITAL SIGNATURE -----
Message = 10
Digital Signature = 10

Enter signature S: 10
Enter message M: 10

----- SIGNATURE VERIFICATION -----
Signature = 10
Message = 10
Calculated M' = 10
The message is authenticate
```

- b. For $p=7$, $q=11$, $e=13$. Find public and private keys using the RSA algorithm. For message, $M=5$, Find the digital signature value. Also show how any receiver of this signature will verify it.

Output:

```
Enter prime number p: 7
Enter prime number q: 11
Enter public key e: 13

----- KEY GENERATION -----
n = 77
phi(n) = 60
Public Key = (13, 77)
Private Key = (37, 77)

Enter message M: 5

----- DIGITAL SIGNATURE -----
Message = 5
Digital Signature = 47

Enter signature S: 47
Enter message M: 5

----- SIGNATURE VERIFICATION -----
Signature = 47
Message = 5
Calculated M' = 5
The message is authenticate
```